CSA0604 – DESIGN AND ANALYSIS OF ALGORITHMS

LAB EXPERIMENTS -SESSION 1

NAME : Shree Ranjane K

REG NO: 192471052

1.Program to Find the First Palindromic String in an Array

```
#include <stdio.h>
#include <string.h>
int main() {
    int n, i, j, palindrome;
    char words[10][50];
  scanf("%d", &n);
for (i = 0; i < n; i++)
        scanf("%s", words[i]);
  for (i = 0; i < n; i++) {
        palindrome = 1;
        j = 0;
 while (j < strlen(words[i]) / 2) {
            if (words[i][j] != words[i][strlen(words[i]) - j - 1]) {
                palindrome = 0;
                break;
            }
            j++;
        }

        if (palindrome) {
            printf("%s", words[i]);
            return 0;
        }
    }

    printf("");
```

```
    return 0;
}
```

INPUT: 5

abc

car

ada

racecar

cool

Output : ada

2. Program to Count Common Elements in Two Arrays

```
#include <stdio.h>
int main() {
    int nums1[] = {2, 3, 2};
    int nums2[] = {1, 2};
    int n = 3, m = 2;
    int answer1 = 0, answer2 = 0;
    int i, j, found;
 for (i = 0; i < n; i++) {
        found = 0;
        for (j = 0; j < m; j++) {
            if (nums1[i] == nums2[j]) {
                found = 1;
                break;
            }
        }
        if (found)
            answer1++;
    }

    for (i = 0; i < m; i++) {
```

```
        found = 0;
        for (j = 0; j < n; j++) {
            if (nums2[i] == nums1[j]) {
                found = 1;
                break;
            }
        }
        if (found)
            answer2++;
    }

    printf("[%d, %d]", answer1, answer2);

    return 0;
}
```

Input :

3

2 3 2

2

1 2

Output: [2 , 1]

3. Program to Find the Sum of Squares of Distinct Counts of All Subarrays

```
#include <stdio.h>
int main() {
    int n, nums[100];
    int i, j, k, count;
    long long sum = 0;
 printf("Enter size of array: ");
    scanf("%d", &n);
```

```
  printf("Enter array elements: ");
   for (i = 0; i < n; i++)
       scanf("%d", &nums[i]);
for (i = 0; i < n; i++) {
       count = 0;
for (j = i; j < n; j++) {
           int found = 0;
 for (k = i; k < j; k++) {
               if (nums[k] == nums[j]) {
                   found = 1;
                   break;
               }
           }

           if (!found)
               count++;

           sum += count * count;
       }
   }

   printf("Output: %lld", sum);

   return 0;
}
```

INPUT :

3

1 2 1

1 2 1

OUTPUT : 15

4. Program to Count Good Pairs in an Array

```
#include <stdio.h>

int main() {
    int n, k, i, j;
    int nums[100];
    int count = 0;

    printf("Enter size of array: ");
    scanf("%d", &n);

    printf("Enter array elements: ");
    for (i = 0; i < n; i++)
        scanf("%d", &nums[i]);

    printf("Enter k: ");
    scanf("%d", &k);

    for (i = 0; i < n; i++) {
        for (j = i + 1; j < n; j++) {
            if (nums[i] == nums[j] && (i * j) % k == 0)
                count++;
        }
    }

    printf("Output: %d", count);

    return 0;
}
```

INPUT :

7

3 1 2 2 2 1 3

2

OUTPUT : 4

5. Program to Find the Maximum Element in an Array

```
#include <stdio.h>

int main() {
    int n, i, max;
    int a[100];

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements: ");
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);

    max = a[0];

    for (i = 1; i < n; i++) {
        if (a[i] > max)
            max = a[i];
    }

    printf("Maximum element = %d", max);

    return 0;
}
```

INPUT :

-10 2 3 -4 5

OUTPUT : 5

6. Program to Sort a List and Find the Maximum Element

```
#include <stdio.h>

void merge(int a[], int low, int mid, int high) {
    int i = low, j = mid + 1, k = 0;
    int temp[100];

    while (i <= mid && j <= high) {
        if (a[i] <= a[j])
            temp[k++] = a[i++];
        else
            temp[k++] = a[j++];
    }

    while (i <= mid)
        temp[k++] = a[i++];

    while (j <= high)
        temp[k++] = a[j++];

    for (i = low, k = 0; i <= high; i++, k++)
        a[i] = temp[k];
}

void mergeSort(int a[], int low, int high) {
    if (low < high) {
        int mid = (low + high) / 2;

        mergeSort(a, low, mid);
        mergeSort(a, mid + 1, high);
        merge(a, low, mid, high);
    }
```

```
}

int main() {
    int n, a[100], i;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    if (n == 0) {
        printf("List is empty");
        return 0;
    }

    printf("Enter elements: ");
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);

    mergeSort(a, 0, n - 1);

    printf("Sorted list: ");
    for (i = 0; i < n; i++)
        printf("%d ", a[i]);

    printf("\nMaximum element: %d", a[n - 1]);

    return 0;
}
```

INPUT :

Test Cases

1. Empty List

Input: 0

Output: List is empty

2. Single Element

Input:

1

5

Output:

Sorted list: 5

Maximum element: 5

3. All Elements Same

Input:

5

3 3 3 3 3

Output:

Sorted list: 3 3 3 3 3

Maximum element: 3

Time Complexity: O(n log n)
Space Complexity: O(n)

7. Program to Find Unique Elements in a List

```
#include <stdio.h>

int main() {
    int n, a[100], unique[100];
    int i, j, count = 0, found;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements: ");
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);
```

```
    for (i = 0; i < n; i++) {
        found = 0;

        for (j = 0; j < count; j++) {
            if (a[i] == unique[j]) {
                found = 1;
                break;
            }
        }

        if (!found)
            unique[count++] = a[i];
    }

    printf("Unique elements: ");
    for (i = 0; i < count; i++)
        printf("%d ", unique[i]);

    return 0;
}
```

INPUT :

8

3 7 3 5 2 5 9 2

OUTPUT :3 7 5 2 9

Space Complexity

O(n) — We use an additional array unique[] that can store up to n elements.

Time Complexity: $O(n^2)$ in the worst case.

8. Sort an array of integers using the bubble sort technique.

```
#include <stdio.h>
```

```
int main() {
    int n, a[100], i, j, temp;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements: ");
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);

    // Bubble Sort
    for (i = 0; i < n - 1; i++) {
        for (j = 0; j < n - i - 1; j++) {
            if (a[j] > a[j + 1]) {
                temp = a[j];
                a[j] = a[j + 1];
                a[j + 1] = temp;
            }
        }
    }

    printf("Sorted array: ");
    for (i = 0; i < n; i++)
        printf("%d ", a[i]);

    return 0;
}
```

Input:

5

5 2 8 1 3

Output:

Sorted array: 1 2 3 5 8

Time Complexity – Big-O

- **Best Case:** O(n) — if the array is already sorted *(with an optimized bubble sort)*.
- **Average Case:** $O(n^2)$
- **Worst Case:** $O(n^2)$ — when the array is in reverse order.

Space Complexity: O(1) because Bubble Sort sorts the array in-place.

9. Program to Search an Element Using Binary Search

```
#include <stdio.h>

int main() {
   int arr[] = {-9, 3, 4, 6, 8, 9, 10, 30};
   int n = 8, key;
   int low = 0, high = n - 1, mid;
   int found = 0;

   printf("Enter key: ");
   scanf("%d", &key);

   while (low <= high) {
      mid = (low + high) / 2;

      if (arr[mid] == key) {
         printf("Element %d is found at position %d", key, mid + 1);
         found = 1;
         break;
      }
      else if (arr[mid] < key)
         low = mid + 1;
      else
         high = mid - 1;
   }
```

```
    if (!found)
        printf("Element %d is not found", key);

    return 0;
}
```

Test Case 1

Input:

10

Output:

Element 10 is found at position 7

Test Case 2

Input:

100

Output:

Element 100 is not found

Time Complexity – Big-O

Best Case: O(1) — element is found at the middle.

Average Case: O(log n)

Worst Case: O(log n)

Space Complexity: O(1).

10. Program to Sort an Array Using Merge Sort

```
#include <stdio.h>

void merge(int a[], int left, int mid, int right) {
    int i = left, j = mid + 1, k = 0;
    int temp[100];

    while (i <= mid && j <= right) {
        if (a[i] <= a[j])
            temp[k++] = a[i++];
        else
```

```
            temp[k++] = a[j++];
    }

    while (i <= mid)
        temp[k++] = a[i++];

    while (j <= right)
        temp[k++] = a[j++];

    for (i = left, k = 0; i <= right; i++, k++)
        a[i] = temp[k];
}

void mergeSort(int a[], int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;

        mergeSort(a, left, mid);
        mergeSort(a, mid + 1, right);
        merge(a, left, mid, right);
    }
}

int main() {
    int n, a[100], i;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements: ");
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);
```

```
    mergeSort(a, 0, n - 1);

    printf("Sorted array: ");
    for (i = 0; i < n; i++)
        printf("%d ", a[i]);

    return 0;
}
```

Input:

5

5 2 8 1 3

Output:

Sorted array: 1 2 3 5 8

Complexity

- **Time Complexity:** O(n log n)
- **Space Complexity:** O(n) for the temporary array.
- **Built-in sorting:** Not used.

Merge Sort is a good choice here because it meets the required O(n log n) time complexity.